

FRTree Planner: Robot Navigation in Cluttered and Unknown Environments with Tree of Free Regions

Yulin Li^{1,*}, Zhicheng Song^{2,*}, Chunxin Zheng², Zhihai Bi², Kai Chen²,
Michael Yu Wang³, *Fellow, IEEE*, and Jun Ma¹

Abstract—In this work, we present FRTree planner, a novel robot navigation framework that leverages a tree structure of free regions, specifically designed for navigation in cluttered and unknown environments with narrow passages. The framework continuously incorporates real-time perceptive information to identify distinct navigation options and dynamically expands the tree toward explorable and traversable directions. This dynamically constructed tree incrementally encodes the geometric and topological information of the collision-free space, enabling efficient selection of the intermediate goals, navigating around dead-end situations, and avoidance of dynamic obstacles without a prior map. Crucially, our method performs a comprehensive analysis of the geometric relationship between free regions and the robot during online replanning. In particular, the planner assesses the accessibility of candidate passages based on the robot’s geometries, facilitating the effective selection of the most viable intermediate goals through accessible narrow passages while minimizing unnecessary detours. By combining the free region information with a bi-level trajectory optimization tailored for robots with specific geometries, our approach generates robust and adaptable obstacle avoidance strategies in confined spaces. Through extensive simulations and real-world experiments, FRTree demonstrates its superiority over benchmark methods in generating safe, efficient motion plans through highly cluttered and unknown terrains with narrow gaps. The open-source project can be found at https://github.com/ly100/navigation_with_tree_of_free_regions.

I. INTRODUCTION

Over the past decades, the field of motion planning in robotics has witnessed significant advancements, which markedly improve the mobility and flexibility of mobile robots when navigating through complex environments [1]–[4]. Despite these achievements, the development of an autonomous navigation system capable of operating efficiently in cluttered and fully unknown environments remains a formidable challenge in research. In such settings, the robot primarily encounters two types of challenges. First, when navigating through narrow passages, the navigation system must assess if a narrow gap is traversable based on the robot’s geometry, and then generate safe and effective motion plans to pass through

it. Otherwise, it may lead to overly conservative maneuvers to bypass accessible narrow passages, or failure to find a feasible path. Second, in fully unknown environments with limited sensor range, the system must autonomously make informed decisions based on local perception information at each replanning phase. This involves selecting intermediate goals, overcoming dead-end situations, and avoiding unforeseen dynamic obstacles, which render it even more challenging in cluttered environments.

In our previous work [5], a bi-level trajectory optimization algorithm is proposed to generate collision-free trajectory by constraining robots with specific geometries to be contained within the free space over the entire optimization horizon. However, this method relies on pre-decomposition in a known environment, which involves sampling points in the free space for extracting free regions until sufficient regions are generated to approximate the entire free space. In cluttered environments with complex obstacle layouts, an excessive number of regions are required to represent the free space. This results in a dense graph with redundant information that complicates the search for a reference path and also increases the computational burden. Additionally, effectively sampling points at narrow gaps poses significant challenge, which subsequently leads to formation of low-quality and non-traversable regions in narrow passage. Essentially, it is the typical case that these challenges render the optimization problem rather difficult to solve, and even lead to failure of convergence in some scenarios. Furthermore, since this method lacks mechanisms to update the graph using local perception information and explore the optimal directions, its deployment in certain circumstances is inherently restricted, for example, when the robot gets trapped in confined spaces or navigates around moving obstacles in high-dynamic environments.

To address all the aforementioned limitations, this paper extends our previous work [5] and proposes FRTree planner for robot navigation in cluttered and unknown environments. The overview of the proposed navigation framework is shown in Fig. 1, and the main contributions of our work are as follows:

- We propose a novel map-free robot navigation framework that effectively exploits the topology of free space by constructing a tree of free regions. This approach facilitates online replanning of safe and efficient goal-directed trajectories in unknown and cluttered environments with limited sensor range.
- Real-time perception information is continuously integrated to expand the tree toward directions that can be explored and transversed, and this allows the robot

*indicates equal contribution.

¹Yulin Li and Jun Ma are with the Division of Emerging Interdisciplinary Areas, The Hong Kong University of Science and Technology, Hong Kong SAR, China (e-mail: yline@connect.ust.hk; jun.ma@ust.hk)

²Zhicheng Song, Chunxin Zheng, Zhihai Bi, and Kai Chen are with the Robotics and Autonomous Systems Thrust, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China (e-mail: zsong469@connect.hkust-gz.edu.cn; czheng739@connect.hkust-gz.edu.cn; zbi217@connect.hkust-gz.edu.cn; kchen916@connect.hkust-gz.edu.cn)

³Michael Yu Wang is with the School of Engineering, Great Bay University, China (e-mail: mywang@gbu.edu.cn)

to effectively select the most viable intermediate goals, mitigate dead-end situations, and avoid dynamic obstacles without relying on a prior map.

- The framework efficiently identifies narrow passages that are traversable, tailored to the robot’s specific geometry. Free regions generated along directions leading to these narrow passages are integrated with a backend geometry-aware, collision-free trajectory optimization. This integration allows for more robust and effective generations of adaptable obstacle avoidance behaviors in highly cluttered spaces.

II. RELATED WORKS

Sampling-based methods have long been favored in motion planning for their efficiency in finding paths in cluttered environments. Techniques like rapidly exploring random tree (RRT) and probabilistic roadmap (PRM) create paths by connecting sampled points in the collision-free space towards the goal. Although these methods offer probabilistic completeness, they lack asymptotic optimality. To address this, extensions such as RRT* [6] and RRT*-Smart [7] have been developed to guarantee optimal solutions provided with enough samples. However, the sampling-based approach requires substantial computational resources, making it impractical for real-time recomputation in dynamic environments. To address this, variants of these methods regenerate feasible paths as conditions change [8], [9]. They typically modify the initial path by continuously refining the search tree when new information comes. Still, a large search tree needs to be maintained and updated at each replanning phase.

Recent improvements in computational power and optimization algorithms have highlighted the potential of optimization-based methods for generating safe and effective trajectories in complex environments [10]–[12]. In cluttered environments with complex obstacle layouts, the spatial decomposition of the free space is widely explored [13]–[15]. Various techniques exist to obtain such decompositions to efficiently approximate the free space [16], [17], enabling effective enforcement of collision avoidance constraints in the subsequent calculation of safe trajectories. In [16], sampled points on the polynomial trajectory are enforced to be within the extracted free region sequence, which is embedded into a minimum snap framework, adding a safety layer to the optimized motion. Extensions of this idea use polyhedral outer representations to express the whole trajectory utilizing the convex hull property. Specifically, control points of Bernstein basis [2], B-spline basis [1], and MINVO basis [11] can be confined within the safe corridor to ensure the entire trajectory’s safety. Yet, these methods only generate free regions along the reference path searched from dilated obstacle information without considering specific robot geometries [16], [18], [19], typically leading to failure to find feasible paths, especially in cluttered environments with narrow passages. To address this, a bi-level trajectory optimization framework has been proposed to consider the robot’s specific geometry, which ensures that the robot remains within the sequence of free regions throughout the optimization process [5].

TABLE I
NOMENCLATURE

Symbols	Descriptions
$\mathcal{P}_p$	Point cloud data perceived at position p
$\mathcal{F}_p$	Set of all feature points extracted at position p
$\mathcal{Q}_{n_i}$	Sequence of free regions stored in node i
$\mathcal{S}_{n_i}$	Set of all child nodes of node i
$\mathcal{R}_{n_i}$	Set of interesting direction grown from node i
$\mathcal{V}$	Set of all free regions marked for visited nodes
$\mathcal{D}$	Set of all free regions marked for dead nodes
$C(\mathcal{Q}) \in \mathbb{R}^3$	Geometric center of a polytopic free region $\mathcal{Q}$
$p_{n_i}^r \in \mathbb{R}^3$	Replan point for node i
$p_s \in \mathbb{R}^3$	Start position
$p_g \in \mathbb{R}^3$	Goal position

In fully unknown settings, maintaining an incrementally fused map, such as an occupancy map or ESDF map, is necessary due to limited sensor range and perception noise. The local reference is regenerated based on the updated map fused with new sensor data, which is then refined into feasible, collision-free trajectories via backend optimization. Although effective and widely applied in many advanced navigation systems [2], [19], [20], this two-step navigation framework can suffer from additional computational burden due to pre-processing and accumulated mapping errors. Alternatively, a graph of connected free regions has the potential to efficiently explore the spatial structure and represent larger portions of the free space compared to discrete grids [5], [21], [22]. However, existing methods either rely on a pre-generated graph of free regions and lack mechanisms to update the graph locally, or their graphs represent each node as a single region generated from a sample point. These approaches do not fully utilize the topology information of the free space to examine traversability and guide exploration, leading to difficulties in finding feasible paths and resulting in conservative maneuvers in narrow and cluttered spaces.

III. METHODOLOGY

This work aims to develop a navigation framework that generates collision-free motion plans for a robot with specific geometry to achieve real-time goal-directed maneuvers in cluttered and fully unknown environments. The proposed navigation system is capable of continuously integrating new sensory information within the constraints of limited sensor range, to replan safe and efficient trajectories toward the intended goal configuration. This entails the system’s proficiency in efficient planning and exploration, selection of the most viable path, generation of safe and dynamically feasible trajectories in narrow spaces, recovery from trapping into local optima, and adaptation to unexpected or changing environments. Specifically, the framework introduces an online replanning mechanism that examines the geometric layout of the free space, as illustrated in Fig. 1, which primarily relies on the iterative execution of three sequential steps at each replanning phase:

- Dynamical construction of the tree of free regions $\mathcal{T}$.
- Intermediate goal selection with updated $\mathcal{T}$.
- Geometry-aware collision-free trajectory optimization.

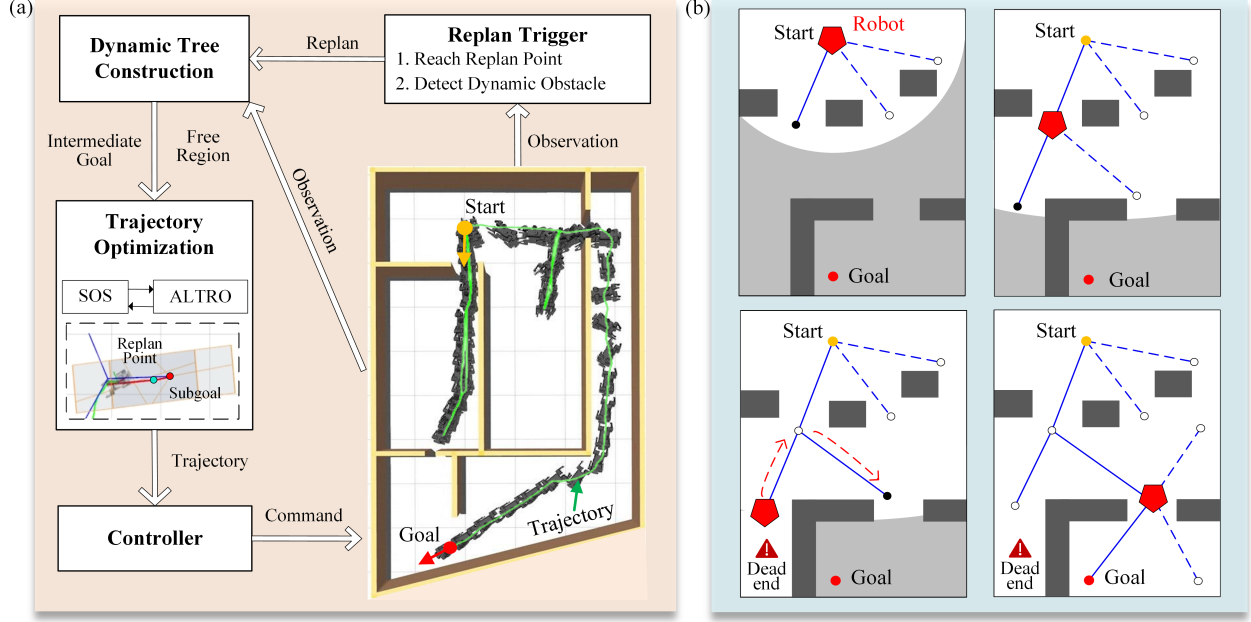


Fig. 1. Overview of the proposed navigation framework. (a) Illustration of the framework pipeline. At each replanning phase, a tree of free regions is dynamically constructed to efficiently embed information about the free space and potential exploration directions. The next feasible and explorable intermediate goal is inferred and fed into the subsequent geometry-aware bi-level trajectory optimization framework to achieve safe and efficient navigation in unknown and cluttered environments with narrow passages and bug traps. (b) Visualization of the navigation process with limited sensor range. As navigation progresses, the free region tree is continuously updated that records visited and dead-end areas. This enables the consistent selection of suitable intermediate goals, ensuring safe and efficient navigation to the destination.

Pertinent notations used in this work are listed in Table I for convenience. Starting from p_s in an initially unknown environment with dense obstacles, we progressively gather more information about the surroundings as the robot navigates the environment. This is achieved by incrementally constructing a tree of free regions $\mathcal{T}$ rooted at p_s . Utilizing this updated graph, we continuously choose appropriate intermediate goals and optimize the local trajectory, and this enables the robot to safely and efficiently reach p_g in complex environments.

A. Dynamic Tree Construction

In this section, we explain the tree structure and how to update it online using real-time perception data. In our proposed framework, we represent a node as a potential direction for exploration from its parent node, together with the geometric information of the free space extracted along that direction. Each node encapsulates a specific path and its associated free regions, enabling an efficient representation of free space while minimizing information overhead compared to conventional occupancy grid maps. Specifically, for each node $n_i \in \mathcal{N}$, we extract a sequence of free regions $\mathcal{Q}_i$ that represents the spatial structure of the exploration direction. A replan point $p_{n_i}^r$ is then associated with n_i as the intermediate goal for trajectory optimization when the node is selected for further exploration. The process of dynamically constructing a tree with such nodes is illustrated in Algorithm 1.

1) *Interesting Directions Extraction*: At the replan point $p_{n_i}^r$ of n_i , we first extract k feature points (shown as the red dots at the obstacle corners in Fig. 2(a)) directly from the point cloud data $\mathcal{P}_{p_{n_i}^r}$ based on the smoothness information

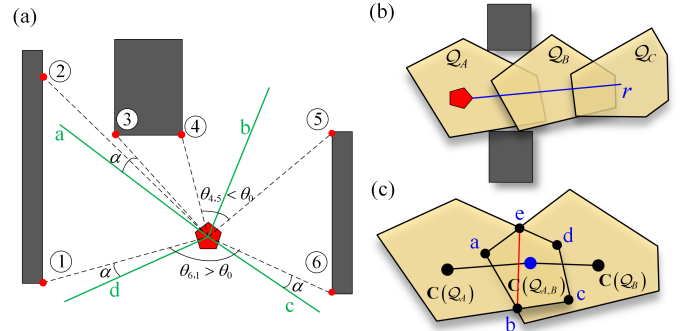


Fig. 2. Illustration of the dynamic tree construction. (a) Visualization of the process for identifying the interesting directions. (b) Depiction of the free regions sequence generation for each node along its destination direction r . (c) Process of pruning infeasible paths at narrow passages. We evaluate the qualities of the free regions $\mathcal{Q}_A$, $\mathcal{Q}_B$, and their intersection $\mathcal{Q}_{A,B}$ to ensure the safe transition from $\mathcal{Q}_A$ to $\mathcal{Q}_B$. We search among $a - e$ (vertices of the intersection $\mathcal{Q}_{A,B}$) to find the shortest line segment be (shown in red) that intersects the reference path from $\mathcal{Q}_A$ to $\mathcal{Q}_B$ (the line segments connecting $C(\mathcal{Q}_A)$, $C(\mathcal{Q}_{A,B})$, and $C(\mathcal{Q}_B)$).

utilizing the algorithm outlined in [23]. Then, r interesting directions (green lines in Fig. 2(a)) are identified by evaluating the relationships between adjacent feature points based on three specific rules. The principle behind these rules is to extract directions that are worth exploring and non-redundant in the surrounding area to represent the topology of collision-free space, which shares similarity with conventional frontier points detection methods used widely in exploration area [24]. The process is depicted in Fig. 2(a). Denote $\theta_{i,j}$ the angle between two adjacent feature points i and j , starting from one feature point, we sequentially evaluate the obstacle information

between two consecutive feature points clockwise and generate all interesting directions with the following rules.

- **Rule 1:** If two adjacent feature points are blocked by a smooth-surfaced obstacle, such as ① → ②, ③ → ④, and ⑤ → ⑥, we do not generate interesting directions between them.
- **Rule 2:** When a sudden change or jump in the depth of obstacle points is identified between two feature points, such as ② → ③, we consider that there may be a potential navigable path between these two points. Therefore, we generate an interesting direction a .
- **Rule 3:** In situations where no obstacles are presented between two consecutive feature points, such as ④ → ⑤ and ⑥ → ①, we further evaluate $\theta_{i,j}$. If this angle exceeds the set threshold θ_0 (typically taking $\pi/2$), we consider it a large unknown area and extend branches on both sides since these two paths could potentially lead to different patterns in an unknown environment. Otherwise, only one direction in the middle of the crack will be generated, considering that one exploration direction is sufficient in this case.

Note that all interesting directions are offset by a bias α towards the free space to avoid obstruction by obstacles, which could lead to poor quality of subsequently extracted regions along these lines.

2) *Dynamic Tree Update:* With the extracted interesting directions at node i , denoted as $\mathcal{R}_{n_i} = \{r_1, r_2, \dots, r_r\}$, we grow the tree $\mathcal{T}$ from n_i to extend r child nodes corresponding to each interesting direction:

$$\mathcal{S}_{n_i} = \{n_{i,1}, n_{i,2}, \dots, n_{i,r}\}.$$

Here, to represent the tree structure, we extend the notation of a node by indicating its parent node in the subscript, e.g., $n_{i,j}$ is the j th child node extending from node i . For the j th node in $\mathcal{S}_{n_i}$, we sequentially extract three overlapping polytopic free regions along the r_j using the decomposition algorithm in [16] and store them in $\mathcal{Q}_{n_{i,j}} = \{\mathcal{Q}_A, \mathcal{Q}_B, \mathcal{Q}_C\}$, as visualized in Fig. 2(b). The replan point of $n_{i,j}$ is set as the geometric center of the middle free region at $C(\mathcal{Q}_B)$. To achieve safe and effective navigation in unknown and cluttered environments with narrow passages, it is crucial to evaluate the quality of the free regions for the robot with specific geometries to traverse, while avoiding re-exploration of previously visited areas or dead-end situations within our mapless framework.

In this sense, we introduce two additional steps to prune nodes from $\mathcal{S}_{n_i}$ that lead to infeasible trajectories or previously visited areas and dead ends. For each $n_{i,j} \in \mathcal{S}_{n_i}$, we first filter out routes that are impassable for our robot, eliminating paths that would inevitably fail if passed to the backend optimizer. This step reduces the unnecessary computational load on the optimizer. Specifically, we calculate the volume of each free region in $\mathcal{Q}_{n_{i,j}}$ with their intersections, and eliminate the node if these volumes are smaller than the robot's volume. Secondly, we observe that even if the intersection of free regions is large enough, the robot may still not be able to smoothly transition from one region to another considering its specific geometry. To address this issue, we

Algorithm 1: Dynamic Tree Update

Input: Tree $\mathcal{T}$, current node n_i , point cloud data $\mathcal{P}_{p_{n_i}^r}$, dead-end regions $\mathcal{D}$, visited regions $\mathcal{V}$
Output: Updated tree $\mathcal{T}$, updated dead-end regions $\mathcal{D}$

- 1 $\mathcal{F}_{p_{n_i}^r} \leftarrow$ Extract k feature points base on $\mathcal{P}_{p_{n_i}^r}$;
- 2 $\mathcal{R}_{n_i} \leftarrow$ Identify r interesting directions from $\mathcal{F}_{p_{n_i}^r}$;
- 3 $\mathcal{Q}_{n_{i,r}}, p_{n_{i,r}}^r \leftarrow$ Generate sequences of polytopic free regions along each $r_r \in \mathcal{R}_{n_i}$ and associate the replan point;
- 4 $\mathcal{S}_{n_i} \leftarrow$ Add each child node of n_i ;
- 5 $\hat{\mathcal{S}}_{n_i} \leftarrow$ Prune non-traversable nodes using $\mathcal{Q}_{n_{i,r}}$ and visited/dead nodes with set $\mathcal{D}$ and $\mathcal{V}$;
- 6 **if** $\hat{\mathcal{S}}_{n_i} = \emptyset$ **then**
- 7 Mark node n_i as dead;
- 8 Update dead-end regions $\mathcal{D}$;
- 9 **end**

define the intersection of $\mathcal{Q}_A$ and $\mathcal{Q}_B$ as $\mathcal{Q}_{A,B}$ and extract all the vertices in $\mathcal{Q}_{A,B}$, as indicated in Fig. 2(c). From these vertices, we calculate the shortest line segment that separates $\mathcal{Q}_A \cup \mathcal{Q}_B$ into two into distinct regions and compare it to the robot's minimum cross-sectional length. This cross-section is considered the necessary path for the robot to traverse from $\mathcal{Q}_A$ to $\mathcal{Q}_B$. If the robot cannot pass through this segment with any posture, we deem the path non-traversable and eliminate it.

Besides, to achieve effective and efficient navigation in unknown environments with limited sensor information, we keep tracking two additional sets, $\mathcal{V}$ and $\mathcal{D}$, to record free regions that represent previously visited and dead-end area. After selecting the next intermediate goal $n_{i,j}$, the free region at $p_{n_{i,j}}^r$ will be added in the set $\mathcal{V}$, and the node will be labeled as visited. For each $n_{i,j} \in \mathcal{S}_{n_i}$, if the replan point $p_{n_{i,j}}^r$ is within $\mathcal{V}$ or $\mathcal{D}$, the respective node will be excluded. Also, we calculate the angle between the interesting direction and the robot's forward direction. If this angle exceeds a predefined threshold, the direction is filtered out to prevent redundant backward exploration.

Following the two pruning steps, the truncated set of $\mathcal{S}_{n_i}$ is:

$$\hat{\mathcal{S}}_{n_i} = \{n_{i,1}, n_{i,2}, \dots, n_{i,l}\},$$

with $l \leq r$. If $\hat{\mathcal{S}}_{n_i} = \emptyset$, we consider n_i as non-extendable and mark it as dead. In this case, the free region at $p_{n_i}^r$ will be stored in the set $\mathcal{D}$. For each remaining child node in $\hat{\mathcal{S}}_{n_i}$, we establish an edge $e_{i,j}$ by sequentially connecting the geometric centers of the regions in $\mathcal{Q}_{n_{i,j}}$ through their intersections, as illustrated in Fig. 2(c). These resulting line segments serve as the reference path for traversing between these two nodes, with the edge length $\ell_{e_{i,j}}$ defined as the cumulative length of these segments. Through this process, both geometrical and topological information of the collision-free space are incrementally encoded in $\mathcal{T}$. This enables robots to continuously select intermediate goals, facilitating efficient and safe navigation towards the destination. The comprehensive procedure is succinctly presented in Algorithm 1.

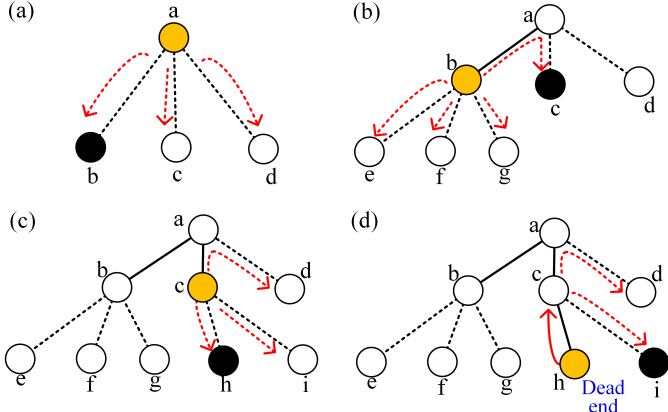


Fig. 3. An example of intermediate goal selection during navigation. At the current node (shown in yellow), we first add all the child nodes and the second-best child node of its parent (if it exists) to a candidate intermediate goal set $\mathcal{M}$ (the red dashed lines). We then select the node with the minimum estimated cost from $\mathcal{M}$ as the current intermediate goal (the black node). Black dashed lines represent unexplored nodes in $\mathcal{T}$, while black solid lines represent visited paths. Notably, in unknown environments, if a dead end is encountered as in (d), the process backtracks (the red solid lines) till it finds the parent node with other feasible nodes for further exploration using the connectivity information of $\mathcal{T}$.

B. Intermediate Goal Selection

In this section, we introduce the criteria for selecting the intermediate goal at each replanning phase. Based on the updated tree $\mathcal{T}$, we adopt a greedy search strategy by choosing the next forwarding node with the minimum cost from a candidate queue $\mathcal{M}$. The cost of choosing a node is divided into two parts. The first part is the distance from the current replan point to the replan point of the selected node (geometric center of the middle free region along that direction), through the free regions corridor. This distance is then added to the straight-line distance from the geometric center of the furthest free region directly to the goal, serving as the esteemed cost-to-go from the selected node. In this strategy, we assume that unexplored and unseen areas are free when calculating the cost-to-go, which helps the robot to converge to the destination efficiently from the current position. Additionally, to handle common bug traps in unknown and cluttered environments, we introduce a backtracking mechanism based on the tree's structure to the next best route from the previous node to continue exploration. We will present the idea using the example process visualized in Fig. 3 for clarity.

Starting from n_a , as shown in Fig. 3(a), the set of all candidate intermediate goals is defined as:

$$\mathcal{M}_a = \{n_b, n_c, n_d\},$$

which is ordered by their corresponding cost:

$$\mathcal{L}_a = \{\ell_{e_a,b} + \ell_{e_b,goal}, \ell_{e_a,c} + \ell_{e_c,goal}, \ell_{e_a,d} + \ell_{e_d,goal}\}.$$

Next, after arriving at the replan point of n_b , a new round of replanning is triggered, extending n_e, n_f, n_g as shown in Fig. 3(b). Instead of only searching among the child nodes of n_b , we also consider the possibility of selecting the parent node's suboptimal child node (n_c in this case), since the limited sensor range may prevent us from seeing the entire obstacle

Algorithm 2: Intermediate Goal Selection

Input: Tree $\mathcal{T}$, robot current position p_r , goal position p_g , current node n_i , dead-end regions $\mathcal{D}$, visited regions $\mathcal{V}$

Output: Updated tree $\mathcal{T}$, updated dead-end regions $\mathcal{D}$, updated visited regions $\mathcal{V}$, intermediate node n_j

```

1 if  $n_i$  is marked as dead then
2    $n_i \leftarrow$  Parent node of  $n_i$ ;
3   while  $n_i$  has no other feasible child node do
4      $n_i \leftarrow$  Parent node of  $n_i$ ;
5   end
6 end
7  $\mathcal{M}_i \leftarrow$  Get candidate intermediate goals of  $n_i$ ;
8  $\mathcal{L}_i \leftarrow$  Compute cost of each candidate node in  $\mathcal{M}_i$ ;
9  $\mathcal{M}_i \leftarrow$  Sort intermediate goals based on  $\mathcal{L}_i$ ;
10  $n_j \leftarrow$  Select intermediate goal with minimum cost from  $\mathcal{M}_i$ ;
11 Mark  $n_j$  as visited;
12 Update visited regions  $\mathcal{V}$ ;
13 return  $n_j$ 

```

layout in one frame. For instance, if a long wall blocks the subsequent path, going back for one step might offer a better cost. The candidate set at n_b is thus:

$$\mathcal{M}_b = \{n_c, n_e, n_f, n_g\},$$

and the cost for traversing from n_b to n_c is:

$$\ell_{b,c} = \ell_{e_b,a} + \ell_{a,c}.$$

Subsequently, as illustrated in Fig. 3(c), the situation at n_c is similar to that at n_b , the only difference is that although $\ell_{a,b} < \ell_{a,d}$, node b has already been explored and marked as visited. The free region recorded at n_b helps us avoid revisiting previously explored areas, thereby preventing redundant operations and enabling more efficient exploratory navigation. Therefore, the candidate set at n_c is:

$$\mathcal{M}_c = \{n_h, n_i, n_d\},$$

from which n_h is chosen at this step.

At n_h , no feasible child nodes are extended by the dynamic graph updating module. the robot is considered to have entered a bug trap, exemplified by the situations in areas A and B of Fig. 4. To address this challenge, we propose an efficient and effective autonomous backtracking mechanism leveraging the constructed $\mathcal{T}$ to escape such dead ends. The backtracking process involves the robot iteratively retracing its steps to parent nodes until it reaches a node with unexplored feasible branches. Subsequently, a new intermediate goal is selected from the remaining candidate set. As illustrated in Fig. 3(d), backtracking to node n_c suffices for further exploration towards nodes n_i and n_d . Node n_i is chosen as the next intermediate goal due to the shorter path length:

$$\ell_{c,i} < \ell_{c,a} + \ell_{a,d}.$$

To this end, the intermediate goal selection algorithm, including the backtracking mechanism, is summarized in Algorithm 2.

C. Collision-Free Trajectory Optimization

With the next forwarding node selected, we aim to generate a safe and effective motion plan to ensure smooth traversing from the current position to the intermediate goal. Specifically,

suppose we are navigating a robot $\mathcal{B}$ from n_i to n_j on $\mathcal{T}$, the following trajectory optimization is formulated:

$$\begin{aligned}
& \underset{(\mathbf{q}_\tau, \mathbf{u}_\tau) \in \mathbb{R}^n \times \mathbb{R}^m}{\text{minimize}} && \phi_T(\mathbf{q}_T) + \sum_{\tau=0}^{T-1} J_\tau(\mathbf{q}_\tau, \mathbf{u}_\tau) \\
& \text{subject to} && \mathbf{q}_{\tau+1} = f(\mathbf{q}_\tau, \mathbf{u}_\tau), \\
& && \mathbf{u}_\tau \in [\mathbf{u}_{\text{lower}}, \mathbf{u}_{\text{upper}}], \\
& && \tau = 0, 1, \dots, T-1 \\
& && \mathcal{W}_\mathcal{B}(\mathbf{q}_\tau) \subseteq \mathcal{Q}_{n_i, j}, \\
& && \tau = 0, 1, \dots, T \\
& && \mathbf{q}_0 = \mathbf{p}_{n_i}^r.
\end{aligned}$$

In this optimization problem, we seek the optimal state-control trajectory $(\mathbf{q}, \mathbf{u})$ over the horizon T , subjecting to dynamic constraints f , control limit constraints, and safety constraints. Notably, the safety constraints enforce that the space occupied by the robot $\mathcal{B}$ at each $\mathbf{q}_\tau$, denoted as $\mathcal{W}_\mathcal{B}(\mathbf{q}_\tau)$, to be contained within the safe corridor from n_i to n_j , i.e., $\mathcal{Q}_{n_i, j}$, which guarantees geometry-aware collision-free maneuvers along the entire trajectory. The goal constraint is only considered in the cost function since the replan point of n_j may not be safe. To accurately model the safety constraints and solve the nonlinear and nonconvex problem effectively, we formulate a Sums-of-Squares (SOS) programming problem to determine the minimum scaling factor for the free region to encompass the robot at a specific configuration [5]. The value and gradient information from this scaling problem is integrated into the augmented Lagrangian iterative linear quadratic regulator (AL-iLQR) based solver ALTRO [25], resulting in an effective and efficient bi-level pipeline to handle the implicit geometry-aware safety constraints with rapid convergence. Detailed implementations of the trajectory optimization algorithm can be found in [5].

IV. RESULTS

In this section, we validate the effectiveness of our proposed framework for various challenging navigation tasks through both simulations and real-world experiments. In simulations, the framework is implemented on an Intel i5-13400F processor. We first evaluate the overall performance of the proposed navigation framework in a maze environment with narrow passages and dead ends, arising from unknown and cluttered settings. Next, we benchmark our method against several baseline methods in a random $15\text{m} \times 5\text{m}$ forest to further highlight the contributions and advantages of our navigation framework in generating safe and efficient navigation behavior in cluttered and narrow space without any prior knowledge of the map. Besides, we deploy the proposed framework on a Unitree GO1 robot, with the entire system running on an Intel NUC13 with an i7-1360P processor, to navigate the robot through an unknown and cluttered indoor environment with dynamic obstacles, showcasing its practicality and robustness in real robotic applications. Finally, we have conducted additional simulations on randomly generated long-distance, narrow forest terrains with various robot shapes navigating through them. These experiments highlight the versatility of

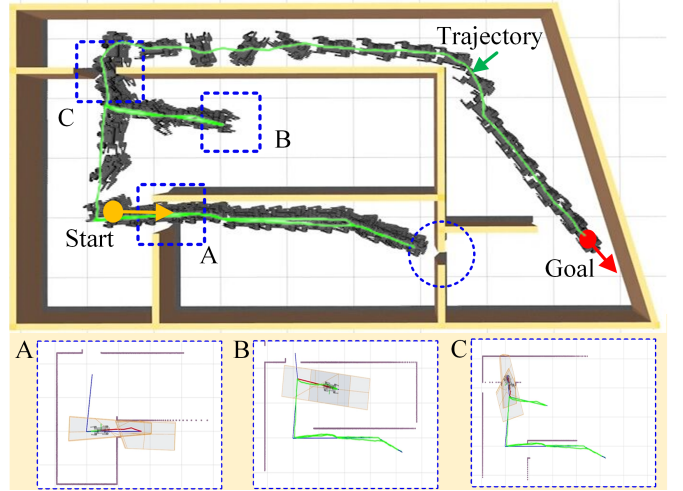


Fig. 4. Performance of our proposed navigation framework in the maze scenario.

The overall trajectory from the start (yellow dot) to the goal (red dot) is visualized with keyframes highlighted. During navigation, the robot successfully overcomes the bug traps in frame B and the blue-circled area in region A, navigates through the narrow passages in frames A and C, and ultimately reaches the goal safely and efficiently.

TABLE II
AVERAGE COMPUTATION TIME FOR PRIMARY MODULES

Step	Time [ms]
Dynamic Tree Construction	0.06
Intermediate Goal Selection	0.08
Trajectory Optimization	118

our method in accommodating different robot geometries and navigating challenging environments. The results of these experiments are available on the project website for further reference. The trajectory optimization problem is solved using ALTRO [25] with the safety constraint handled implicitly as described in our previous research [5]. During the bi-level solving iterations, the certifiable safety SOS programming problem is solved using the conic programming solver COPT [26].

A. Simulations

1) *Maze*: In this subsection, we assess the performance of our proposed navigation framework in an unknown environment, as depicted in Fig. 4. Our goal is to command a $0.6\text{m} \times 0.4\text{m}$ quadruped from the start to the goal, marked by yellow and red dots, respectively. We visualize the entire navigation process, highlighting key moments that demonstrate common challenges in cluttered and unknown environments. Our framework relies on the available perception data and the dynamically updated free regions tree $\mathcal{T}$ to select the feasible direction with the shortest estimated cost to the goal. Consequently, the robot attempts two shorter routes to reach the goal. However, the first path is blocked by a passage narrower than the robot's width while the other path ends in

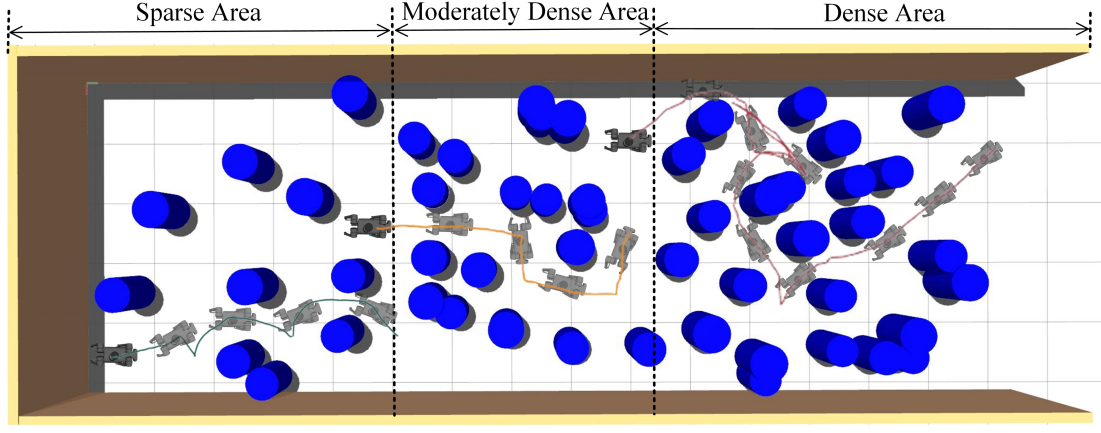


Fig. 5. Visualization of three selected trajectories generated from our methods in the forest environment (15 m $\times$ 5 m). Our method efficiently and safely navigates through narrow terrains of varying obstacle densities exploiting the dynamically constructed free region tree considering specific robot geometries with no prior map.

TABLE III
COMPARISON RESULTS OF OUR METHOD WITH RRTX [9] AND FASTER [2] IN THE FOREST ENVIRONMENT

		RRTX	Faster	Ours
Sparse Area	Complete Rate	0.7	0.8	1
	Length Scale	1.46	1.45	1.12
	Collision Free	✓	✓	✓
Moderately Dense Area	Complete Rate	0	1	1
	Length Scale	N/A	1.73	1.65
	Collision Free	N/A	✗	✓
Dense Area	Complete Rate	0	1	1
	Length Scale	N/A	1.25	1.1
	Collision Free	N/A	✗	✓

a dead end. Using the dynamic tree updating rules outlined in Sec. III-A, the robot detects these dead-end situations, triggering a backtracking mechanism that allows it to retreat from these bug traps. Notably, due to the exploiting of the geometric relationship between the tight-fitted robot and free spaces in our navigation framework, the robot flexibly adapts its posture to navigate through narrow passages, as shown in frames A and C of Fig. 4. The computation times of the primary modules during navigation are recorded in Table II, and the entire navigation system operates in real-time at around 10 Hz.

2) *Forest*: In the forest environment, we further compare our proposed framework with RRTX [9] and Faster [2]. As shown in Fig. 5, to demonstrate the effectiveness and robustness of our algorithm in cluttered environments, we randomly generate obstacles with three different densities throughout the forest: 0.4 obstacles/m², 0.7 obstacles/m², 1 obstacle/m². For fairness, we set the same sensor range in the implementations of RRTX and Faster. In the three areas of varying complexity, we conduct five experiments each from different start and goal points for every method, with the completion rate, average navigation path length, and safety record in Table III. Each successful arrival at the goal is recorded as a complete, and to standardize path length, we define the length scale as the ratio of the actual path length to the straight-line distance between each start and goal configuration. Typical navigation paths of

our method is visualized in Fig. 5. In moderately dense and dense areas, RRTX often fails to rewire a kinodynamically feasible path and sometimes gets stuck oscillating between two routes. Faster, as a state-of-the-art navigation framework, generally succeeds in reaching the goal in all three scenarios. It maintains a local occupancy map around the robot and uses the Jump Point Search (JPS) method to continuously search for a path to the goal. It then generates a safe corridor along this path and optimizes the robot's trajectory within the corridor. When the local map is larger than the actual map, Faster relies on the map information to escape bug traps. Both RRTX and Faster simplify the robot's shape and inflate obstacles, which in dense areas can lead to discarding the shortest feasible path for the robot's shape or failing to find a feasible route, resulting in detours and unsafe situations.

Our method does not rely on maintaining a dense map and takes the specific shape of the robot into account. Instead, it dynamically updates a free region tree to effectively identify passable or impassable narrow gaps based on the robot's shape. In unknown environments, it continuously selects the nearest feasible path, overcomes bug traps, and optimizes a safe and effective trajectory based on the robot's tight-fitting geometry, generating non-conservative and flexible obstacle avoidance maneuvers in cluttered environments.

B. Real-World Experiment

In this section, we deployed our framework on a unitree Go1 robot to test its performance in a cluttered indoor environment. In the experiment, we command the robot to traverse an indoor area of 5 m $\times$ 6 m with several randomly placed obstacles. We use the onboard MID360 LiDAR to perceive the point cloud data for dynamic tree updating. As illustrated in Fig. 6, the free region tree was continuously updated during the navigation process. This dynamic updating allowed the system to select suitable intermediate goals, enabling the robot to adjust its position and orientation as needed. As a result, the robot was able to successfully navigate around both static and dynamic obstacles, demonstrating its ability to adapt to changes in the environment and maintain a safe path toward its destination.

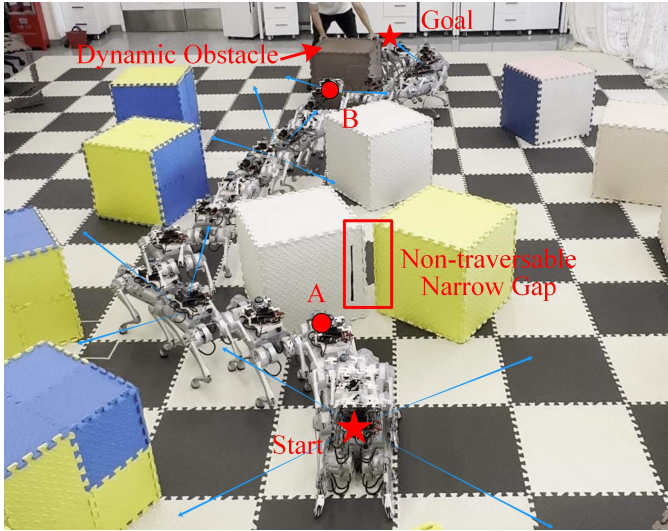


Fig. 6. Visualization of the overall trajectory in the real-world experiment. With dynamically constructed free region trees (the directions are visualized as blue arrows), the robot continuously selects suitable paths, successfully identifies impassable narrow gaps, and navigates around both static and changing environments to reach the goal.

Our system demonstrates the essential capability to mitigate real-time challenges and ensure reliable performance in complex and unpredictable settings, highlighting the effectiveness and robustness of our approach in real-world applications.

V. CONCLUSION

In this paper, we extend our bi-level trajectory optimization algorithm [5] with an online replanning module for real-time, geometry-aware collision avoidance in cluttered and unknown environments. Our framework incrementally constructs a tree of free regions, efficiently representing the geometrical and topological information of the free space. During each replanning phase, exploratory paths are extended, and sequences of free regions are extracted to update the tree. The shortest feasible direction is continuously selected towards the target configuration, enabling safe and efficient navigation while adapting to environmental changes. Extensive experiments demonstrate the capability of the proposed framework in handling complex obstacle layouts and unknown terrains, ensuring safe and reliable navigation for robots with specific geometries.

REFERENCES

- [1] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and efficient quadrotor trajectory generation for fast autonomous flight," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3529–3536, 2019.
- [2] J. Tordesillas, B. T. Lopez, M. Everett, and J. P. How, "Faster: Fast and safe trajectory planner for navigation in unknown environments," *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 922–938, 2022.
- [3] L. Huber, J.-J. Slotine, and A. Billard, "Avoidance of concave obstacles through rotation of nonlinear dynamics," *IEEE Transactions on Robotics*, vol. 40, pp. 1983–2002, 2024.
- [4] Z. Tian, Z. Liu, X. Zhou, and W. Shi, "Unguided self-exploration in narrow spaces with safety region enhanced reinforcement learning for ackermann-steering robots," in *2024 IEEE International Conference on Mobility, Operations, Services and Technologies (MOST)*, 2024, pp. 260–268.

- [5] Y. Li, C. Zheng, K. Chen, Y. Xie, X. Tang, M. Y. Wang, and J. Ma, "Collision-free trajectory optimization in cluttered environments using sums-of-squares programming," *IEEE Robotics and Automation Letters*, 2024.
- [6] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.
- [7] F. Islam, J. Nasir, U. Malik, Y. Ayaz, and O. Hasan, "RRT*-Smart: Rapid convergence implementation of RRT towards optimal solution," in *IEEE International Conference on Mechatronics and Automation*, 2012, pp. 1651–1656.
- [8] I. Becerra, H. Yervilla-Herrera, E. Antonio, and R. Murrieta-Cid, "On the local planners in the RRT* for dynamical systems and their reusability for compound cost functionals," *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 887–905, 2022.
- [9] M. Otte and E. Frazzoli, "RRTX: Asymptotically optimal single-query sampling-based motion planning with quick replanning," *The International Journal of Robotics Research*, vol. 35, no. 7, pp. 797–822, 2016.
- [10] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 3259–3278, 2022.
- [11] J. Tordesillas and J. P. How, "MADER: Trajectory planner in multiagent and dynamic environments," *IEEE Transactions on Robotics*, vol. 38, no. 1, pp. 463–476, 2022.
- [12] X. Xiao, B. Liu, G. Warnell, and P. Stone, "Toward agile maneuvers in highly constrained spaces: Learning from hallucination," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1503–1510, 2021.
- [13] Y. Li, X. Tang, K. Chen, C. Zheng, H. Liu, and J. Ma, "Geometry-aware safety-critical local reactive controller for robot navigation in unknown and cluttered environments," *IEEE Robotics and Automation Letters*, vol. 9, no. 4, pp. 3419–3426, 2024.
- [14] C. Toumeh and A. Lambert, "High-speed planning in unknown environments for multirotors considering drag," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 7844–7850.
- [15] J. Park, D. Kim, G. C. Kim, D. Oh, and H. J. Kim, "Online distributed trajectory planning for quadrotor swarm with feasibility guarantee using linear safe corridor," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4869–4876, 2022.
- [16] S. Liu, M. Watterson, K. Mohta, K. Sun, S. Bhattacharya, C. J. Taylor, and V. Kumar, "Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-D complex environments," *IEEE Robotics and Automation Letters*, vol. 2, no. 3, pp. 1688–1695, 2017.
- [17] C. Toumeh and A. Lambert, "Voxel-grid based convex decomposition of 3d space for safe corridor generation," *Journal of Intelligent & Robotic Systems*, vol. 105, no. 4, p. 87, 2022.
- [18] L. Huber, J.-J. Slotine, and A. Billard, "Fast obstacle avoidance based on real-time sensing," *IEEE Robotics and Automation Letters*, vol. 8, no. 3, pp. 1375–1382, 2023.
- [19] C. Toumeh and D. Floreano, "High-speed motion planning for aerial swarms in unknown and cluttered environments," *IEEE Transactions on Robotics*, vol. 40, pp. 3642–3656, 2024.
- [20] H. Oleynikova, Z. Taylor, R. Siegwart, and J. Nieto, "Safe local exploration for replanning in cluttered unknown environments for microaerial vehicles," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1474–1481, 2018.
- [21] J. Ji, Z. Wang, Y. Wang, C. Xu, and F. Gao, "Mapless-planner: A robust and fast planning framework for aggressive autonomous flight without map fusion," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 6315–6321.
- [22] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Science Robotics*, vol. 8, no. 84, p. eadf7843, 2023.
- [23] J. Zhang, S. Singh *et al.*, "LOAM: Lidar odometry and mapping in real-time," in *Robotics: Science and Systems*, vol. 2, no. 9, 2014, pp. 1–9.
- [24] A. Batinovic, T. Petrovic, A. Ivanovic, F. Petric, and S. Bogdan, "A multi-resolution frontier-based planner for autonomous 3d exploration," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4528–4535, 2021.
- [25] T. A. Howell, B. E. Jackson, and Z. Manchester, "ALTRO: A fast solver for constrained trajectory optimization," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2019, pp. 7674–7679.
- [26] D. Ge, Q. Huangfu, Z. Wang, J. Wu, and Y. Ye, "Cardinal Optimizer (COPT) user guide," <https://guide.coap.online/copt/en-doc>, 2023.